Batch Intake - Setup Guide

Create Truv verification orders automatically from CSV files. Three commands to set up a sandbox test, then staff save a file into a folder. **You do not need to read any code.**

BEFORE YOU START

Two things to have ready

1. **Your Truv API keys.** Truv Dashboard → **Development** → **API Keys**. You need the **Client ID** and the **Access Secret**. Start with your **sandbox** keys.
2. **A computer that stays on.** A server, or a desktop that does not sleep - not a laptop that gets closed at night.

You also need **Python 3.8 or newer**. Nothing to install beyond that: the tool uses only what ships with Python.

Who does what

One-time setup	A technical person. Going live also needs a scheduled export from your case system and an owner for the keys and folders.
Daily use	Any staff member. Save a CSV into a folder. No technical skill.
Ongoing	Someone who checks `failed/`, or reads the optional summary email.

> **The one thing that catches everyone.** There is a single Truv API address. Sandbox and production are decided by **which keys you paste**, not by a different URL. So "am I in sandbox?" is answered only by which keys are in your config file. Every Truv Access Secret says which it is: production secrets begin with `prod-`, sandbox secrets with `sandbox-`. The tool reads that prefix and **refuses to run if it disagrees with the `environment` line in `config.ini`**, so pasting production keys under `environment = sandbox` stops the tool instead of quietly emailing real applicants. It also refuses to touch production until a sandbox test has passed - please leave that check switched on.

START ON YOUR COMPUTER

The kit works the same on Windows, Mac and Linux. Only the first steps differ.

	WINDOWS	MAC	LINUX
Double-click launcher	`START-HERE-windows.bat`	`START-HERE-mac-linux.command`	`START-HERE-mac-linux.command`
Python command	`py -3`	`python3`	`python3`
Open a terminal here	In File Explorer, click the address bar, type `cmd`, press Enter	Open Terminal, type `cd `, drag the folder in, press Enter	Right-click in the folder → Open in Terminal
Scheduler	Task Scheduler	`python3 truv_intake.py schedule`, or cron	cron

The launcher runs `setup`, then `doctor`, and offers to run `selftest`. Commands in this guide are written with `python3`. **On Windows, type `py -3` instead.**

> **If the launcher is blocked.** The launchers are not signed, so both systems may stop a downloaded copy. **Mac:** click Done, then System Settings → Privacy & Security → Open Anyway - or open a terminal in the folder and type `bash START-HERE-mac-linux.command`. **Windows:** on "Windows protected your PC", click More info → Run anyway.

SETUP - THREE COMMANDS

Open a terminal in the tool's folder and run these in order. Do not skip step 2.

1 Set up

```
python3 truv_intake.py setup
```

Asks for your keys (and, optionally, your mail server), tests the keys against Truv immediately, and creates the folders. Re-running it is safe - press Enter to keep any existing answer.

2 Check, then prove it works

```
python3 truv_intake.py doctor
python3 truv_intake.py selftest
```

`doctor` must show **PASS** on every line; anything that fails tells you exactly what to fix. `selftest` then creates **one real order** and reads it back. Confirm it appears in your Truv dashboard - this is what unlocks production.

3 Schedule it

`setup` prints the exact line for your operating system. It looks like this:

```
# Mac / Linux — run `crontab -e` and add:
30 * * * * cd /path/to/folder && /usr/bin/python3 truv_intake.py run
```

Mac alternative: `python3 truv_intake.py schedule` installs an hourly macOS job with your paths filled in. It runs while you are logged in; for a server with nobody logged in, IT installs it as a LaunchDaemon. Keep the folder outside Documents, Desktop and Downloads.

Windows: Task Scheduler → Create Task → Trigger: Daily, repeat every 1 hour → Action: *Start a program* → the program is your `python.exe`, arguments `truv_intake.py run`, "Start in" is the tool's folder. Tick **Run whether user is logged on or not**.

That is the whole setup. From here, staff drop CSV files into `input/` and the tool does the rest. For schedulers and monitoring, the run exits `0` when everything went through, `2` when a row failed or a file was rejected, and `1` when it could not run.

GOING LIVE

1. In `config.ini`, replace both keys with your **production** keys *and* change `environment = sandbox` to `environment = production`. **Change them together** - the tool checks that they agree and will stop if they do not.
2. Run `doctor` once more. It reports the environment it read from the key, not the one you typed.

The API address does not change. Only the keys and that one label - and a half-finished switch halts the tool rather than creating real orders by accident.

DAILY USE - FOR STAFF

1. Make a CSV, one applicant per row.
2. **Save it into the `input` folder.**
3. Done. The next scheduled run processes it.

Column names. Case, spaces and hyphens do not matter (`First_Name`, `LAST NAME`). Common alternatives are recognized too - Given Name, Surname, Case ID, Case #, Email Address, Phone Number and others; the README lists them all. For any other name, map it in `config.ini` under `[columns]` (for example `first_name = Applicant First`) instead of changing your export.

File formats. Comma-, semicolon- and tab-separated files are read, as are Windows Excel's plain CSV and "Unicode Text". Blank rows are ignored. Extra columns are not sent and not copied into any file the tool writes. The tool refuses a file with *two columns of the same name* and an Excel workbook (`.xlsx`, even if renamed) - re-save it as CSV UTF-8. The template's two sample rows use `example.com` addresses; the tool refuses those rows, so delete them.

OUTCOME	WHERE IT GOES	WHAT IT MEANS
All rows fine	`processed/`	Every row became an order.
Some rows bad	`processed/`	The good rows became orders. The bad ones are written to `failed/` as a `.failed-rows.csv` with the reason on each line - fix them and move that file into `input/`.
Already ordered	`processed/`	Every row had been sent before. Nothing new was created.
Nothing could be sent	`failed/`	The file itself is wrong: wrong columns, a workbook, or more than 3,000 rows - or every row failed. The run log and email say which.

> Two guarantees. **One bad row never blocks the rest** - the good rows are sent, and the bad ones come back in a small file with the reason on each line, ready to fix and drop back in. And **the same applicant is not ordered twice by accident**: the tool remembers everyone it has submitted, keyed on the person and the case, so re-dropping a file creates nothing new. This is strongest when every row has a `case_number` - Truv then also recognizes the order reference, even if the tool's ledger is lost.

How big can a file be?

Up to **3,000 rows**. Your account can create 3,000 orders an hour, and one file is sent in a single request, so a larger file is rejected with instructions rather than half-sent - split it across runs.

CSV columns

COLUMN	REQUIRED	WHAT IT IS
`first_name`	**Yes**	Applicant first name.
`last_name`	**Yes**	Applicant last name.
`case_number`	No	Your case ID. Becomes the order reference and the duplicate key. **Strongly recommended** - without it, duplicates are caught only by the tool's own ledger.
`email`	No	Applicant email. If you fill this in, **Truv emails the applicant.** Leave it blank if you do not want that.
`phone`	No	Any format - `(415) 555-4193` is fine. Reduced to 10 digits.
`staff_name` / `staff_email`	No	The staff member who owns the order. They receive Truv's order notifications.
`cc_emails`	No	Extra people to copy. Separate several with `;`. Maximum 15.
`force_new`	No	Put `yes` to create a new order for the same applicant on purpose - redeterminations, retries, an expired order. The new order's reference gets the date and time added. Without it, a repeat is skipped as already ordered.

Need a blank file? `python3 truv_intake.py template`. To check a file before committing to it - reporting every problem, creating nothing and sending nothing - run `python3 truv_intake.py validate yourfile.csv`.

THE EMAILS

Email is **optional and off until you set it up**. When on, the tool connects to your mail server (checking its TLS certificate) as well as to Truv.

Run summary, after every run, to everyone in `to_addrs`:

- `Truv order intake — 12 created, 0 failed, 0 rejected (2026-07-31 14:30:00)`
- `Truv order intake — ran, nothing to do (2026-07-31 15:30:00)`

That second one matters: if summaries stop arriving, the scheduler or the machine is down - check the newest file in `logs/`.

Per-staff confirmation, after each file: every `staff_email` and CC receives one email listing only their own lines and whether each became an order. It carries the case number and name only - **never any verification results**.

GETTING RESULTS BACK

By default no verification results come back to you - completed verifications are reviewed in the Truv dashboard, and there is no endpoint to secure. The tool does keep applicant names on your machine: the ledger, copies of processed and failed files, results files and logs. They are readable only by the tool's account on Mac and Linux; restrict the folder on Windows, and set `retention_days` to delete old copies.

If you want results delivered into your own systems, that needs a **webhook**: a URL on your servers that Truv calls when an order finishes. See the companion guide, *Webhooks & Results Retrieval*.

If you verify assets, collect two kinds of report. Each employer or account connection has its own PDF. The assets report is a separate, single report for the whole order, retrieved by its ID. A process that only collects the per-connection PDFs misses it. The companion guide shows both. Collect results when the `order-finalized` event arrives - not on earlier status events.

```
python3 truv_intake.py webhook add https://your.host/webhooks/truv
python3 truv_intake.py webhook history     # did Truv reach us? last 30 days
python3 truv_intake.py fetch ORDER_ID      # the JSON
python3 truv_intake.py fetch ORDER_ID --pdf
```

A BETTER OPTION IF YOU OWN THE FORM

If you control the application form that produces the CSV, you can create the order **at the moment the applicant submits** instead of waiting for the next run.

```
Today:   form submitted → row into CSV → wait up to an hour → order created
Direct:  form submitted → order created in seconds → row into CSV as a record
```

The CSV does not go away. It stops being a queue and becomes the daily record of what already happened - a better use for it, and it still catches anything the live call missed. `direct_example.py` in the kit is a complete implementation. Send each applicant through one path only - the direct path does not read the batch tool's ledger.

IF SOMETHING GOES WRONG

WHAT YOU SEE	WHAT TO DO
`PROBLEM … WHAT TO DO:`	Do the thing it says. Every error names its own fix.
Summary emails stopped	The scheduler or the machine is down. Check `logs/`.
`HTTP 401` or `HTTP 403`	Wrong keys - or sandbox keys with `environment = production`. Re-run `setup`.
A file went to `failed/`	Open the summary email. It lists every bad row. Fix and re-drop.
"already ordered" messages	Working as intended. That applicant was submitted before - use `force_new` for a deliberate repeat.
Nothing happens at all	Run `python3 truv_intake.py doctor`.

Need help? Send your Truv contact the file from `failed/` with its `.results.csv`, plus the newest file from `logs/`. **These contain applicant names** - remove them first, or use a channel approved for applicant data. **Never send your `config.ini` or your API keys.**

> **Two files to protect.** `config.ini` holds your API secret; the tool restricts it automatically on Mac and Linux, and on Windows you should limit it to the service account under Properties → Security. And `state/ledger.csv` is what prevents duplicate orders - do not delete it. Without it, rows with a case number are still caught by Truv, but rows without one would be ordered again.